Project Writeup - Cynthia

Game Engine: 2 + 4 + 6 = 12

1. Using lambda to pass game-specific collision-detection-skipping-layer over 2 independent engine unit (from Game to EngineRuntime to EngineCore-System-CollisionSystem) [2]
2. Static Platform + one-way(allowing jump from bottom to top) platform collision resolution system + optimization [4]
3. Chain Physics for tail [6]

Game: 9

1. Clearup code and implement Gravity [1]
2. Set and tuning the values of game constants and more [4]
3. Flower and petals structures [1]
4. Flower rotation animation [2]
5. TailTip and FlowerTrigger “collision detect” [1]

Game Engine

1. Static One-way Platforms - Total 13 hrs for all the subtasks

1. onUpdateBegin
   - regular vertical velocity and acceleration updates
   - updates if latch
   - updates if unlatch
2. collision detect() optimization
   - layer-level skipping-filter when latch
   - entity-level skipping-filter when airborne
   - discard
3. onUpdatePostCollisionDetection – still empty; needs to filter collisions to top-landing only
4. resolve
   - player-platform specific resolution
5. onUpdatePostCollisionResolution
   - at the moment player change from airborne to latch
   - update viewPoint

1.1 Using lambda to pass data over 2 independent engine units

- If playerGrounded, game should skip collision detection between player and all the platforms. Problem: If using playerGround state to condition the skipping of detection in CollisionSystem, GameEngine will be dependent on Game, which shouldn't happen.
- A meaningful optimization on collision detection is done by skip player-platforms detection when playerGrounded.
- But there are unnecessary calculations: if playerGrounded, there will be a general player-position-update due to gravity in onUpdateBegin() in every frame, and then player's position will be set back to original position at starting of the frame later on. This is because regular update happens first, and skipping happens later per frame. I was thinking of how to skip regular update on gravity if the beginning. Thought about ways to skip regular update by adding if conditions, or

adding a STATE and switch STATE for player, or create two archetypes and switch assigned archetypes to player. But then realized I should implement the simplest functionality first, and this optimization could be trivial. 3hrs.

- Tried to add a filter function in iGame but realized that Engine_core has no access to Game. It has the layer of Engine_runtime in between. So instead, pass lambda as a parameter into collision detection. Spent quite some time on understanding lambda declare syntax, `std::function`, and how lambda passes template parameters over two architect layers. Eventually changed from template to std::function due to template's strange declare requirements.

2.1 One-way-through static platform implementation

2.2 Player & one-way-static-platform collision detection and collision resolution. Optimization of skipping collision detection when player latched

- Spliting work into sections in Runtime.cpp. There are many cases to consider for each function about collision: from latch to fall, just fall from air onto a platform, falling in air, jumping from bottom to top of platform and across the platform, etc.

- I want the platform to be one-way on collision: player can jump from bottom to top going through the middle of platform, but should not fall when latch. So after finding collisions pairs in detect(), I could remove pairs in the case player is jumping from down to top in onUpdatePostCollisionDetection, but this way will require another round of search. To have an optimized performance, I added another condition of `skipEntityLevelCollisionCheck(entityA, entityB))` to discard the collision detection of player and platform when player is jumping up though the middle of platform.
- Debug the problem: when player falling from platform and has some horizontal overlap with platform in the process, and press jump key at this time, the player will be set back to the top of the platform, but it

doesn't supposed to be. This is because the condition check for skipEntityLevelDetection wasn't complete.

- Bug: `playerPrevBottom < platform.top() && player.vy() > 0`
- Fix:

```
bool isLanding =
    playerPrevBottom >= platform.top() &&
    player.bottom() <= platform.top() &&
    player.vy() <= 0;

return !isLanding;
```

- Spent too much time on this one, maybe I should've let AI do the planning for me

3. Tail - 9 hrs + some untracked time for understanding math and thinking

- Referencing paper: Advanced Character Physics by Thomas Jakobsen
  - `newPosition = currPosition + velocity + acceleration * dt ^ 2`
  - same as: newPosition = currPosition + momentum + gravity
    - velocity and momentum here can be roughly seen as: currPosition - prevPosition

- Chain Physics does not fit in ECS that much, since it does not store position, velocity, acceleration but stores prev and curr position instead. So need to add cumtomer onDraw to Renderer.cpp by lambda.
- avoid sqrt for calculating correction in Tail's update for optimization
- When tail share the same GRAVITY as player, it appears to be too light with large amplitude movement. So I make tail a unique TAIL_GRAVITY to control the sense of heaviness. But then the tail will always be underneath the player even when player falls, does not have the animation affect of following after player. So create two gravity for the tail: LATCH_TAIL_GRAVITY and UNLATCH_TAIL_GRAVITY and switch them accordingly.

Game

1. Remove redundant code and make it still compile, general planning, implementing gravity, input keys - 3.5 hrs

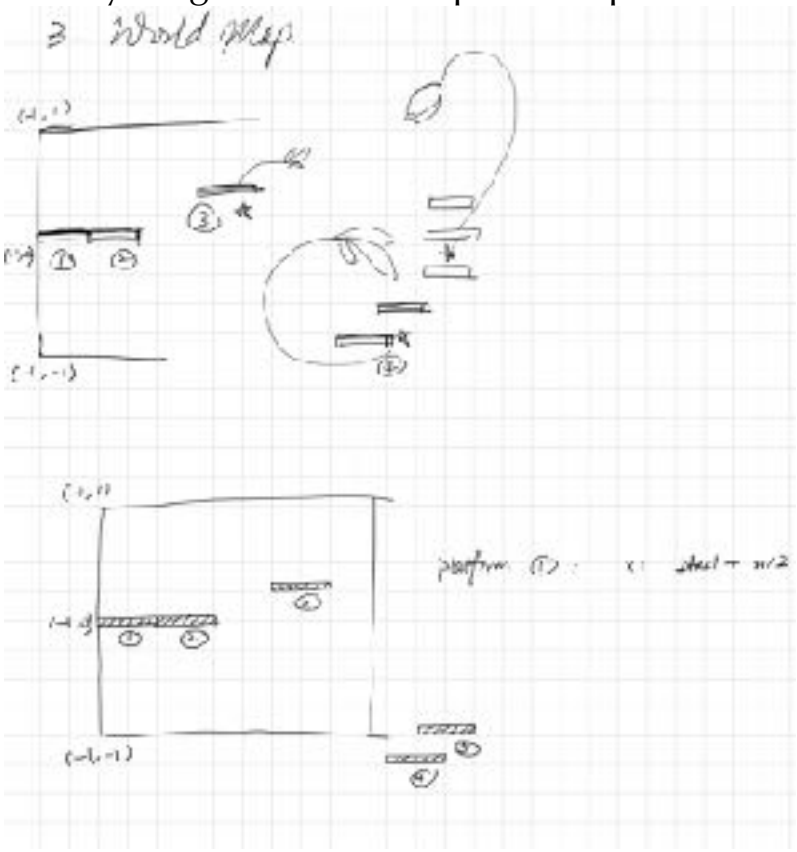
- debug: after creating platform for testing the game, I found that if falling condition of `if (player.x() < platform.left() || player.x() > platform.right()) latchedToPlatform = INVALID_ENTITY_ID;` is misleading. It marks the player as airborne when to users it truly looks like player is still latched to platform. So I changed to:

```
if (player.right() < platform.left() ||
player.left() > platform.right())
latchedToPlatform = INVALID_ENTITY_ID;
```

2. Set and tuning the values of game constants and more [4]

2.1 set up the world and platforms. Tuning the value controlling platform positions and width and height, make the platform's location looks making sense.

- modify magic number for platform positions. Play test.



2.2 tuning value for flower triggers position, setup petals' length, rotation and position away from center, the position of tail root to player, etc.

2.3 create and setup sprites, changing the width and heights, debug segfault

- debug: a garbage data is passed as ptr and caused Seg Fault. Add a validation for SpriteManager::getSprite.

2.4 Play test and find a good balance of jump velocity, tail's latch and unlatch gravities.

3. Flower and petals structures [1]

- similar data oriented idea: create petals as Entity and store it and positions into vector, and use ID / index to represent the collection of petals, or locate the target data.

4. Flower rotation animation [2]

- The intuitive way of create animation was to utilize animation system, and interpolate the rotation progress. But when implementing it, I realized that this was a little conflict with the Entity constructed flower: each petal is an independent Entity, and they supposed to have their movement, so there isn't a sprite sheet to interpolate animation progress. So instead of animation system, I add the functionality on PostKinematics to create the desired effect.
- use cos and sin to create the effect that petals are rotate around center of flower. Petals have some opacity so multiple flower petals overlapped can be seen.

5. rotation of petals and "collision" resolution- [1]

- I was thinking rotation animation could be a rotation of petal entities, but then realize the animation system does not know much about origin of things. It interpolate assets instead. So create an animation update per frame moving the petal entities. I spent some time changing between the two systems, but eventually chose to not use animation system.

- can use interpolation for petals' rotation speed, but this functionality does not involve sprites. Could be a future TODO. Also, each petal's angle could be varied to make more esthetic looks.
- The asset is one single petal and ideally many petals will form a flower. But the petal shape and angle should be more diversed.
- modify flower position and petal width, height, etc.

In general, I found myself spent a lot of time on design and thinking of detailed optimization instead of writing the code. It is good to think about it, but thinking for too long would make it hard to achieve the tasks. I also realize that sometimes I don't have to write the most optimized solution in the beginning, there can always be iterations of codes in the future.